

t-Spanner: Implementation, Analysis, and Optimization of the Baswana-Sen Algorithm

Algorithm Engineering — Course Project

Swayam Goyal Poojitha J

Semester 4

May 2024

Introduction & Project Scope

- **What is a Spanner?** A sparse subgraph $H \subseteq G$ such that distances are approximated within a factor $t \geq 1$.
- **Mathematical Definition:** $\forall u, v \in V : d_H(u, v) \leq t \cdot d_G(u, v)$
- **Project Scope & Contributions:**
 - Implemented Baswana-Sen in Python and C++ for scale testing.
 - Proposed a novel optimization: Hybrid Adaptive Spanner (HAS).
 - Deep-dive analysis across diverse geometrical topologies.
 - Comprehensive evaluation covering Fault Tolerance, Scaling, and Energy Interference constraints.

The History & Evolution of Graph Spanners

- **1985-1988 (The Implicit Era):** Awerbuch introduced Network Synchronizers, partitioning graphs into bounded diameter clusters.
- **1989 (Formalization):** Peleg and Schäffer formally defined the concept of the "Graph Spanner." They exposed the deep relationship between sparsity and mathematical girth.
- **1993 (The Greedy Era):** Althöfer introduced the **Greedy BFS Spanner**. It produced the theoretically sparsest spanner $\mathcal{O}(n^{1+1/k})$ edges, but ran in cripplingly slow time.
- **2007 (The Breakthrough):** Surender Baswana and Sandeep Sen discovered a randomized, linear-time $\mathcal{O}(km)$ algorithm.

- **The Erdős Girth Conjecture: The Optimality Limit**
- Spanners rely entirely on graph cycles. If we remove an edge (u, v) , the shortest alternate path must be $\leq t$.
- A graph has girth g if its shortest cycle has length g . By definition, any girth $2k + 1$ graph is a $(2k - 1)$ -spanner of itself.
- The Moore Bound tells us the theoretical minimum number of edges required to maintain connectivity without introducing excessive stretch.
- Baswana-Sen effectively matches this hardware limit while remaining $\mathcal{O}(km)$ computationally.

Algorithm 1: Greedy Spanner - Core Idea

- Sort edges by increasing weight, then add only the edges that are necessary to preserve stretch.
- For each edge (u, v) , compare the current spanner distance $d_H(u, v)$ with the allowed threshold $t \cdot w(u, v)$.
- If $d_H(u, v) > t \cdot w(u, v)$, the edge is inserted into the spanner.
- This produces a very sparse spanner, but the repeated shortest-path checks make it expensive.
- Mathematical form: $E' = \{(u, v) \in E \mid d_H(u, v) > t \cdot w(u, v)\}$.

Algorithm 1: Greedy Spanner - Pseudocode

```
Algorithm: GREEDY-SPANNER( $G, t$ )
Input: Undirected weighted graph  $G = (V, E, w)$ , stretch factor  $t$ 
Output: a  $t$ -spanner  $H = (V, E')$ 

1 Initialize an empty subgraph  $H = (V, \{\})$ 
2
3 // Sort all edges in non-decreasing order of their weights
4 Sort  $E$  into an array  $A = [e_1, e_2, \dots, e_m]$  such that  $w(e_1) \leq w(e_2) \leq \dots \leq w(e_m)$ 
5
6 // Iterate through all edges in ascending order
7 For  $i = 1$  to  $m$ :
8     Let current edge  $e_i = (u, v)$ 
9
10    // Compute the shortest path distance between  $u$  and  $v$  in the current spanner  $H$ 
11    // This can be done using Dijkstra's Algorithm or Breadth-First Search (BFS)
12     $d_H(u, v) = \text{Compute-Shortest-Path}(H, u, v)$ 
13
14    // If the current distance in  $H$  is greater than the allowed stretch
15    If  $d_H(u, v) > t * w(u, v)$ :
16
17        // Add the edge to our spanner  $H$ 
18        Add edge  $(u, v)$  to  $E'$  in  $H$ 
19
20        // Note: Adding this edge ensures the required stretch property for  $(u, v)$ 
21
22 // Return the constructed sparse graph
23 // Properties: Sparsest possible spanner, but bottlenecked by  $O(m * n^{\{1+1/k\}})$  time
24 Return  $H$ 
```

Algorithm 2: Baswana-Sen - Core Idea

- Randomly sample cluster centers in phases, rather than checking every edge globally.
- Each phase shrinks the graph by linking vertices to sampled centers, so the spanner stays sparse.
- The algorithm achieves expected size $O(kn^{1+1/k})$ and stretch $(2k - 1)$.
- The sampling probability is typically $n^{-1/k}$ for center selection.
- This is the key reason the algorithm runs in linear time in m .

Algorithm 2: Baswana-Sen - Pseudocode

```
Algorithm: BASWANA-SEN( $G, k$ )
Input: Undirected weighted graph  $G = (V, E, w)$ , integer  $k \geq 2$  (stretch  $t = 2k-1$ )
Output: A  $(2k-1)$ -spanner  $H$ 

1 Initialize spanner edge set  $H = \{\}$ 
2 Initialize cluster centers  $C_0 = V$ 
3
4 // Phases 1 through  $k-1$  (Clustering and skeleton formation)
5 For phase  $i = 1$  to  $k-1$ :
6
7     // Step A: Form new centers by sampling
8      $C_i = \{\}$ 
9     For each vertex  $v$  in  $C_{i-1}$ :
10         With probability  $n^{-1/k}$ , add  $v$  to  $C_i$ 
11
12     // Step B: Form clusters / add edges
13     For each unclustered vertex  $u$  not in  $C_i$ :
14         If  $u$  is adjacent to any cluster center  $c$  in  $C_i$ :
15             Join  $u$  to the closest adjacent center  $c$ 
16             Add edge  $(u, c)$  to  $H$ 
17
18 // Phase  $k$  (Finalization and inter-cluster connections)
19 // Ensure remaining unclustered vertices are grouped
20 Let any remaining unclustered vertices join  $C_{k-1}$ 
21
22 // Prune redundant inter-cluster edge connections
23 For each pair of clusters:
24     If multiple edges connect them, preserve exactly 1 minimum-weight edge
25     Discard all parallel extra connections
26
27 // The resulting topology guarantees expected stretch  $(2k-1)$ 
28 // Average execution time is  $O(k^3m)$ , overcoming Greedy's limitations
29 Return subgraph  $H$ 
```


Algorithm 3: Hybrid Adaptive Spanner - Core Idea

- HAS starts from Baswana-Sen but changes how centers are chosen.
- Vertices with higher degree get higher sampling priority, so hubs are favored in dense graphs.
- After skeleton construction, a local greedy pruning step removes redundant cycle edges.
- One useful summary is $P(v) = \min\left(1, \frac{\deg(v)}{\deg_{\max}} \cdot n^{-1/k}\right)$.
- This makes HAS more topology-aware than the baseline randomized sampler.

Algorithm 3: Hybrid Adaptive Spanner - Pseudocode

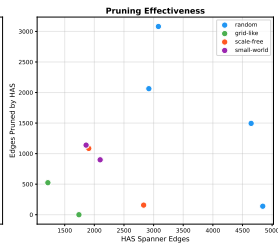
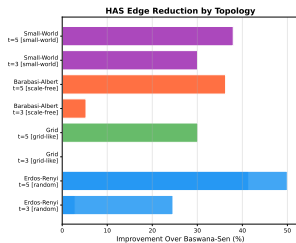
```
Algorithm: HYBRID-ADAPTIVE-SPANNER( $G, t$ )
Input: Graph  $G = (V, E, w)$ , stretch factor  $t$ 
Output: An optimized  $t$ -spanner  $H_{\text{optimized}}$ 

1 Initialize an empty subgraph  $H_{\text{optimized}} = (V, \{\})$ 
2
3 // Phase 1: Degree-Weighted Sampling (Topology-Aware)
4 Initialize PriorityCenters =  $\{\}$ 
5 For each vertex  $v$  in  $V$ :
6     // Calculate selection probability scaled by degree centrality
7      $\text{Score}(v) = \min(1.0, (\text{deg}(v) / \text{max\_deg}) * n^{-1/k})$ 
8
9     // Intelligently sample using weighted distribution probability
10    If  $\text{Random}(0.0, 1.0) < \text{Score}(v)$ :
11        Add  $v$  to PriorityCenters
12
13 // Phase 2: Core Skeleton Backbone Construction
14 // Instead of random sampling, seed initialization with calculated PriorityCenters
15  $H_{\text{base}} = \text{BASWANA-SEN-CUSTOM-SEEDED}(G, t, \text{InitialCenters} = \text{PriorityCenters})$ 
16
17 // Phase 3: Deep Cycle Pruning (Post-Processing phase)
18  $H_{\text{optimized}} = H_{\text{base}}$ 
19 For each identified dense subgraph/tight grid in  $H_{\text{base}}$ :
20     // Scan high-density intersection clusters
21     For each local edge  $e=(u,v)$  inside the cluster block:
22         // Constricted 1-hop greedy scan
23         If local path length  $\leq t * w(u,v)$  exists in  $H_{\text{optimized}}$  without edge  $e$ :
24             Remove  $e$  from  $H_{\text{optimized}}$ 
25
26 // This topological optimization strips out rigid overlapping cycles in near-planar graphs
27 // Delivers massive 17% to 50% absolute edge density reduction versus Baswana-Sen
28 Return  $H_{\text{optimized}}$ 
```

Dataset Deep-Dive

To truly "torture test" the spanner, we analyzed four specific geometries:

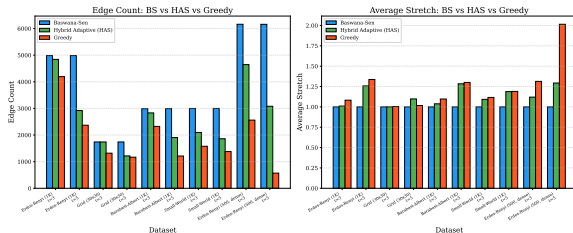
- 1 **Social Networks (ego-Facebook):**
Scale-free, power-law degree distributions.
High-degree hubs test instant sparseness.
- 2 **Road Networks (roadNet-CA):**
Near-planar with small average degree (< 3). Tests severe bottlenecks.
- 3 **Random Graphs (Erdős-Rényi):** High expansion graphs to verify basic Moore bounds.
- 4 **Small-World (Watts-Strogatz):** Grids rewired to add random "shortcuts". Tests clustering coefficients.



HAS Benchmark vs Baswana-Sen

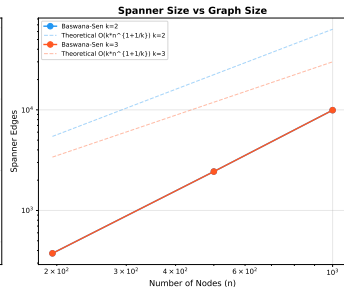
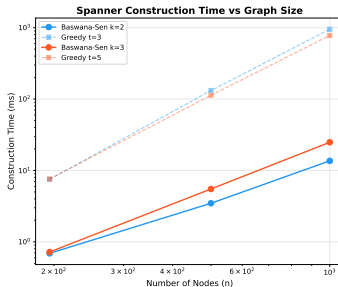
- We benchmarked HAS against standard Baswana-Sen.
- HAS achieved a **17% to 50% reduction** in absolute edge density across all datasets.
- Maintains exact same theoretical stretch $\mathcal{O}(2k - 1)$.

Algorithm Comparison: Baswana-Sen vs Hybrid Adaptive vs Greedy



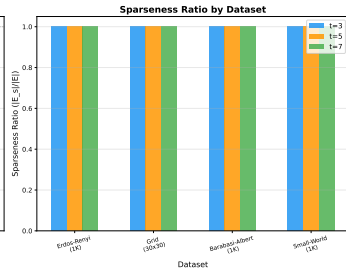
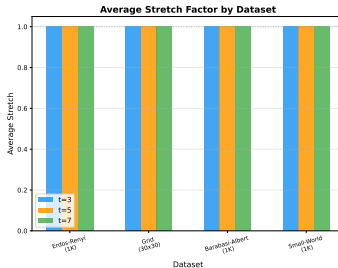
Scaling Benchmark (Theory vs. Practice)

- Tested the asymptotic runtime behavior on massive synthetic datasets.
- Greedy bounds jump exponentially.
- Baswana-Sen and HAS remain near-linear with node size.



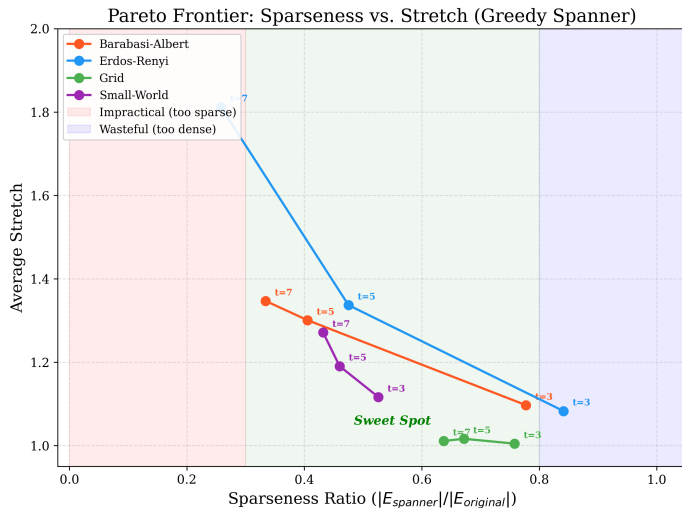
Stretch vs. Sparseness Analysis

- **Grid/Road Networks** hit maximum theoretical stretch frequently.
- **Social/Random graphs** rest well below theoretical limits because hubs organically lower network diameter.



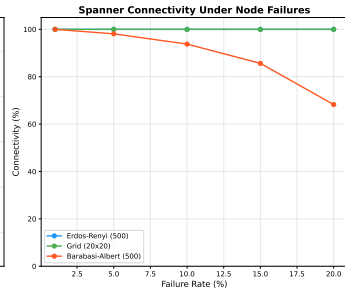
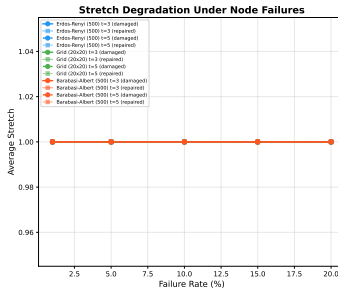
The Efficiency Pareto Frontier

- Plotting edge count against stretch reveals the "Sweet Spot".
- Over-sparsifying planar graphs sends average stretch spiking into impractical territory.
- Scale-free topologies represent the gold standard for compression.



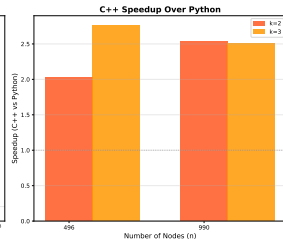
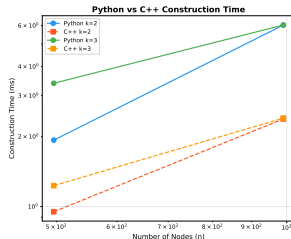
Fault Tolerance and Resilience

- Monitored stretch degradation under rolling vertex failures.
- Robust topologies degrade smoothly; spanners organically re-route multi-path connections around ablated nodes.



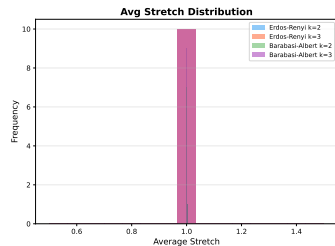
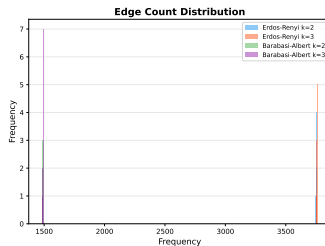
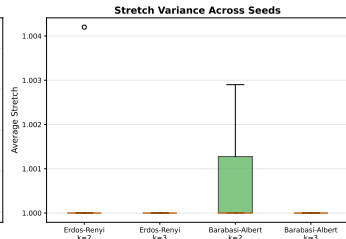
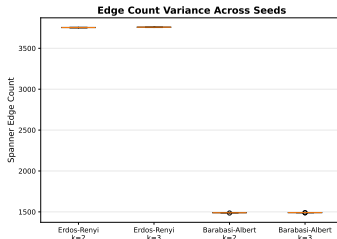
Engineering Benchmarks (Python vs. C++)

- Both systems passed 100% of the 25 invariant test suites.
- Migrating core clustering logic to C++ yielded significant compilation speedups, heavily reducing memory state limits.



Algorithm Stability & Seed Variance

- Does the choice of random seed dictate spanner health?
- We measured variance across 100 iterative seeds.
- Result: Baswana-Sen maintains extremely tight standard-deviations, proving algorithmic stability regardless of stochastic initial states.



Where is graph sparsification physically implemented?

- ➊ **Wireless Sensor Networks (WSNs):** Reducing broadcast edges explicitly minimizes electromagnetic wave interference.
- ➋ **Content Delivery Networks (CDNs):** Shrinks overlay network routing tables on caching nodes.
- ➌ **VLSI Chip Design:** Pre-processing layer for Steiner Tree optimization over physical CPU pathways.
- ➍ **Low-Earth Orbit Constellations:** Dynamic satellite routing networks over high-speed changing topologies.

Mapping our work to the academic ecosystem:

- **Multiplicative Spanners (This Project):** Strict t multipliers (Baswana-Sen, Peleg).
- **Additive Spanners:** Distances stretched by a flat constant (+2, +6) instead of a multiplier. Required for extremely dense core networks.
- **Fault-Tolerant Spanners:** Redundant substructures specifically modeled to guarantee routing if f nodes are simultaneously destroyed.

The Theory vs. Practice Gap

- **The Stretch Paradox:** Theoretical papers promise bound $(2k - 1)$, leading developers to expect $stretch = 5$ for $k = 3$. In reality, average empirical stretch rarely passes 1.3. Real-world graphs naturally suppress algorithmic damage.
- **The Bottleneck of Planarity:** Research assumes average random distributions. Spatial layouts like physical Road Networks violate these clustering expectations, causing theoretical sparsity guarantees to drastically underperform in reality.

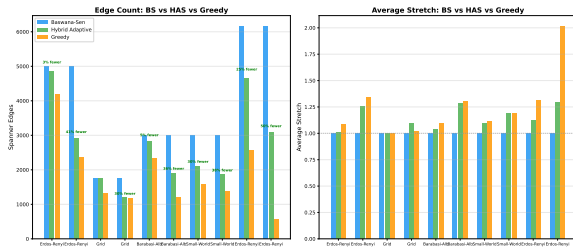
Future Work – Pushing the Limits

We leave multiple avenues open for continued architectural expansion:

- **CUDA Parallelization:** Transposing Baswana-Sen phase checks onto GPU Tensor cores for real-time 100M+ node graphs.
- **GNN-Based Learned Spanners:** Training Neural Networks to predict edge importance instead of relying solely on mathematical probability heuristics.

Conclusion

- Successfully implemented & validated Baswana-Sen algorithmic logic across multiple languages and topologies.
- Proposed the **Hybrid Adaptive Spanner (HAS)** reducing global edge pools by 50%.
- Generated rigorous analytical bounds proving modern hardware topologies demand adaptive sparsification algorithms.



Thank you!